

Fundamentals of Data Structures

Project 2: Normal A+B with Binary Search Trees



Author:

Date: 2026/04/07

2025-2026 Spring & Summer Semester

Table of Contents

Chapter 1: Introduction	3
Chapter 2: Algorithm Specification	3
2.1. Data Structures	3
2.2. Main Program Sketch	3
2.3. Tree Construction (Two-Pass)	4
2.4. Inorder Traversal (Iterative)	4
2.5. Binary Search for Target B	5
2.6. Finding All Valid Pairs	5
2.7. Preorder Traversal (Iterative)	6
2.8. Summary of Complexities	6
Chapter 3: Testing Results	7
3.1. Test Case Table	7
3.2. Sample Output Verification	7
Chapter 4: Analysis and Comments	8
4.1. Time Complexity Analysis	8
4.2. Space Complexity Analysis	8
4.3. Design Decisions	8
4.4. Possible Improvements	9
Appendix: Source Code (in C)	9
Declaration	13

Chapter 1: Introduction

A **Binary Search Tree (BST)** is a binary tree in which every node satisfies the following properties: all keys in its left subtree are strictly less than the node's key, and all keys in its right subtree are greater than or equal to the node's key. Both subtrees are themselves BSTs.

Given two binary search trees T_1 and T_2 and a target integer N , the task is to find all pairs (A, B) such that A comes from T_1 , B comes from T_2 , and $A + B = N$. The results must be printed in ascending order of A , with duplicate equations removed. Finally, the preorder traversal sequences of both trees must be output.

The input format is non-standard: instead of providing an insertion sequence, each node is described by its key value and the index of its parent node (with -1 denoting the root). The program must reconstruct the tree structure from this parent-based representation.

The key algorithmic challenges are:

1. **Tree construction:** determining whether each node is a left or right child based on the BST property.
2. **Efficient pair search:** avoiding $O(n_1 \times n_2)$ brute force by exploiting the sorted nature of BST inorder traversals and binary search.
3. **Deduplication:** handling duplicate keys that may appear in the same tree.

Chapter 2: Algorithm Specification

2.1. Data Structures

The primary data structure is a statically allocated array of nodes. Each node stores a key and the indices of its left and right children (-1 if absent):

```
1 typedef struct {
2     long long key;
3     int left;
4     int right;
5 } Node;
```



Static arrays `Node T1[MAXN]` and `Node T2[MAXN]` (with `MAXN = 200005`) are declared globally to avoid stack overflow. An auxiliary array `parent[]` records each node's parent index during input.

2.2. Main Program Sketch

The main program coordinates the execution by combining tree construction, inorder traversal to collect sorted keys, pair finding, and preorder traversal.

```

1 function Main():
2    $T_1, \text{root}_1 \leftarrow \text{BuildTree}(n_1, \text{input}_1)$ 
3    $T_2, \text{root}_2 \leftarrow \text{BuildTree}(n_2, \text{input}_2)$ 
4    $\text{len}_1 \leftarrow \text{CollectInorder}(T_1, \text{root}_1, \text{keys}_1)$ 
5    $\text{len}_2 \leftarrow \text{CollectInorder}(T_2, \text{root}_2, \text{keys}_2)$ 
6   FindPairs( $\text{keys}_1, \text{len}_1, \text{keys}_2, \text{len}_2, N$ )
7   PrintPreorder( $T_1, \text{root}_1$ )
8   PrintPreorder( $T_2, \text{root}_2$ )

```

2.3. Tree Construction (Two-Pass)

Since a child node's parent may appear later in the input, we use a two-pass approach:

- **Pass 1:** Read all `(key, parent_index)` pairs and initialize each node's `left` and `right` to `-1`.
- **Pass 2:** For each node i , if `parent[i] == -1`, mark it as root. Otherwise, compare node i 's key with its parent's key to determine placement:

```

1 for  $i = 0$  to  $n - 1$ 
2   if  $\text{parent}[i] = -1$  then
3     |  $\text{root} = i$ 
4   else
5     |  $p = \text{parent}[i]$ 
6     | if  $T[i].\text{key} < T[p].\text{key}$  then
7       |  $T[p].\text{left} = i$       // left child
8     | else
9       |  $T[p].\text{right} = i$     // right child (includes equal keys)
10    | end
11  end
12 end

```

The rule $k \geq \text{parent.key} \rightarrow \text{right child}$ follows directly from the BST definition stated in the problem (right subtree contains keys $\geq$ the node's key).

Time Complexity: $O(n)$ — one pass to read, one pass to connect.

2.4. Inorder Traversal (Iterative)

To collect all keys from a BST in ascending order, we perform an iterative inorder traversal using an explicit stack. This avoids potential stack overflow for deep (skewed) trees with up to 2×10^5 nodes.

```

1 Input: tree array T[], root index, output array out[]
2 Initialize stack (top = 0), set cur = root, count = 0
3 while cur ≠ -1 or top > 0
4   while cur ≠ -1      // push all left descendants
5   | push cur onto stack; cur = T[cur].left
6   end
7   pop cur from stack
8   out[count++] = T[cur].key      // visit node
9   cur = T[cur].right      // process right subtree
10 end
11 return count

```

Time Complexity: $O(n)$ — each node is pushed and popped exactly once.

2.5. Binary Search for Target B

Given the sorted key array of T_2 , we use standard binary search to check whether $B = N - A$ exists:

```

1 Input: sorted array arr[], length len, target value target
2 left = 0, right = len - 1
3 while left ≤ right
4   mid = left + (right - left) / 2      // overflow-safe
5   if arr[mid] = target then return 1
6   else if arr[mid] < target then left = mid + 1
7   else right = mid - 1
8 end
9 return 0

```

Time Complexity: $O(\log n_2)$ per query.

2.6. Finding All Valid Pairs

Iterate through the sorted keys of T_1 . For each unique A , compute $B = N - A$ and binary search in T_2 's sorted keys:

```

1 Collect sorted keys: keys1 = inorder(T1), keys2 = inorder(T2)
2 for  $i = 0$  to len1 - 1
3   if  $i > 0$  and keys1[i] = keys1[i - 1] then continue      // skip duplicates
4    $A = \text{keys1}[i]$ ,  $B = N - A$ 
5   if findKey(keys2, len2, B) then

```

```

6 | | store  $(A, B)$  in result
7 | end
8 end

```

Since inorder traversal produces ascending keys, duplicate values are adjacent and can be skipped by a simple comparison with the previous element. The results are automatically sorted by A .

Time Complexity: $O(n_1 \log n_2)$ total.

2.7. Preorder Traversal (Iterative)

We use an explicit stack for preorder traversal as well. The key trick is to push the right child first so that the left child is processed first (LIFO order):

```

1 Input: tree array  $T[]$ , root index
2 Push root onto stack; set first = 1
3 while stack is not empty
4 | Pop cur from stack
5 | if first then print key; first = 0
6 | else print “ “ + key
7 | if  $T[\text{cur}].\text{right} \neq -1$  then push right child
8 | if  $T[\text{cur}].\text{left} \neq -1$  then push left child
9 end
10 Print newline

```

Time Complexity: $O(n)$.

2.8. Summary of Complexities

Operation	Time Complexity	Space Complexity
Build Tree (two-pass)	$O(n)$	$O(n)$
Inorder Traversal	$O(n)$	$O(n)$ (stack + output)
Find All Pairs	$O(n_1 \log n_2)$	$O(n_1)$ (result storage)
Preorder Traversal	$O(n)$	$O(n)$ (stack)
Overall	$O(n_1 \log n_2 + n_1 + n_2)$	$O(n_1 + n_2)$

Chapter 3: Testing Results

3.1. Test Case Table

Case ID	Description	Purpose	Expected Result	Actual Behavior	Possible Cause	Status
TC-01	Sample Input 1 (8 nodes + 7 nodes, N=36)	Verify correct tree construction, pair finding with duplicates (two 18s in T1), and preorder output.	true; 3 equations (15+21, 16+20, 18+18); preorder as given.	Output matches expected exactly.	N/A	pass
TC-02	Sample Input 2 (5 nodes + 3 nodes, N=40)	Verify correct handling of no-solution case.	false; preorder: "10 5 2 7 15" and "15 10 20".	Output matches expected exactly.	N/A	pass
TC-03	Single-node trees: T1={5}, T2={3}, N=8	Edge case: minimal trees with a valid pair.	true; "8 = 5 + 3"; preorder: "5" and "3".	Output matches expected.	N/A	pass
TC-04	Negative keys: T1 contains -10, T2 contains 50, N=40	Verify handling of negative key values.	true; "40 = -10 + 50" among results.	Correctly finds pairs involving negative numbers.	N/A	pass
TC-05	All duplicate keys in T1: five nodes all with key=7, T2={3}, N=10	Verify deduplication: only one equation should be printed.	true; "10 = 7 + 3" printed once.	Only one equation printed despite 5 duplicate keys.	N/A	pass
TC-06	Skewed BST: T1 is a right-skewed chain of 10 nodes (1,2,...,10)	Verify iterative traversal handles deep trees without stack overflow.	Correct inorder (1..10) and preorder (1,2,...,10).	Output correct; no stack overflow.	N/A	pass

3.2. Sample Output Verification

Sample Input 1:

```
1 true
2 36 = 15 + 21
3 36 = 16 + 20
4 36 = 18 + 18
5 15 13 12 14 17 16 18 18
6 20 16 13 18 25 21 28
```

Sample Input 2:

```
1 false
2 10 5 2 7 15
3 15 10 20
```

Both outputs match the expected results from the problem specification exactly.

Chapter 4: Analysis and Comments

4.1. Time Complexity Analysis

The overall time complexity is $O(n_1 \log n_2 + n_1 + n_2)$, which simplifies to $O(n_1 \log n_2)$ when n_1 and n_2 are of similar magnitude.

- **Tree construction** takes $O(n)$ per tree (two linear passes).
- **Inorder traversal** takes $O(n)$ per tree.
- **Pair finding** iterates over n_1 keys and performs a binary search ($O(\log n_2)$) for each, giving $O(n_1 \log n_2)$.
- **Preorder traversal** takes $O(n)$ per tree.

In the **worst case**, if T_2 is a skewed BST (all nodes in a chain), a BST-based search on the tree structure would degrade to $O(n_2)$ per query, making the total $O(n_1 \times n_2)$. However, our implementation avoids this by performing binary search on the **sorted array** obtained from inorder traversal, guaranteeing $O(\log n_2)$ per query regardless of tree shape.

4.2. Space Complexity Analysis

The space usage is $O(n_1 + n_2)$:

- Two node arrays: $O(n_1 + n_2)$.
- Two sorted key arrays: $O(n_1 + n_2)$.
- Result arrays: $O(n_1)$ in the worst case.
- Traversal stack: $O(\max(n_1, n_2))$ in the worst case (skewed tree).

4.3. Design Decisions

1. Two-pass tree construction. A single-pass approach assumes parent nodes are read before their children. Since the input order is arbitrary (a child may appear before its parent), we first read all data, then connect edges in a second pass. This avoids the need for deferred linking or dynamic reordering.

2. Iterative traversals. With n up to 2×10^5 , a skewed BST would have depth 2×10^5 . Recursive traversal would overflow the call stack (typically limited to $\sim 10^4$ frames). Iterative traversal with an explicit stack eliminates this risk.

3. Array-based binary search instead of BST search. Searching directly on the BST structure has $O(n)$ worst-case per query for skewed trees. By extracting a sorted array via inorder traversal and using standard binary search, we guarantee $O(\log n)$ per query.

4. `long long` for keys. The key range is $\pm 2 \times 10^9$. While `int` (32-bit) can hold this range, computing $A + B$ or $N - A$ might overflow `int` when N is close to $\pm 2 \times 10^9$. Using `long long` prevents this issue.

4.4. Possible Improvements

- **Two-pointer technique:** After obtaining both sorted arrays, use two pointers (one starting from the beginning of `keys1` and one from the end of `keys2`) to find all pairs in $O(n_1 + n_2)$ time, which is asymptotically better than $O(n_1 \log n_2)$.
- **Hash table:** Store all keys of T_2 in a hash set for $O(1)$ average lookup, reducing pair finding to $O(n_1)$ expected time.
- **Memory optimization:** The sorted key arrays duplicate information already present in the node arrays. An in-place approach using tree traversal with BST search could save memory at the cost of worst-case time complexity.

Appendix: Source Code (in C)

```
1  #include <stdio.h>
2
3  #define MAXN 200005
4
5  // Each node stores key and child indexes in static arrays.
6  typedef struct {
7      long long key;
8      int left;
9      int right;
10 } Node;
11
12 // Global buffers avoid stack overflow when n is close to 2e5.
13 Node T1[MAXN], T2[MAXN];
14 long long keys1[MAXN], keys2[MAXN];
15 long long N;
16 long long resultA[MAXN], resultB[MAXN];
17 int parent1[MAXN], parent2[MAXN];
18 int workStack[MAXN];
19 int resultCount = 0;
20
21 // Iterative inorder traversal: output keys in ascending order for BST.
22 int collectInorder(Node t[], int root, long long out[]) {
```



```

23     // top is the current size of manual stack.
24     int top = 0;
25     // cur walks down left edges, then backtracks.
26     int cur = root;
27     // count is output length of sorted keys.
28     int count = 0;
29
30     // Standard iterative inorder template.
31     while (cur != -1 || top > 0) {
32         // Push all left descendants.
33         while (cur != -1) {
34             workStack[top++] = cur;
35             cur = t[cur].left;
36         }
37         // Visit node at stack top.
38         cur = workStack[--top];
39         out[count++] = t[cur].key;
40         // Then process right subtree.
41         cur = t[cur].right;
42     }
43
44     // Return number of valid keys written.
45     return count;
46 }
47
48 // Standard binary search on sorted key array.
49 int findKey(long long arr[], int len, long long target) {
50     // Closed interval [left, right].
51     int left = 0;
52     int right = len - 1;
53
54     while (left <= right) {
55         // Overflow-safe midpoint.
56         int mid = left + (right - left) / 2;
57         // Found exact B value.
58         if (arr[mid] == target) return 1;
59         // Discard left half if target is larger.
60         if (arr[mid] < target) left = mid + 1;
61         // Otherwise discard right half.
62         else right = mid - 1;
63     }
64
65     // Not found in T2 key set.
66     return 0;
67 }

```

```

68
69 // Iterative preorder traversal (root-left-right) with strict spacing control.
70 void printPreorder(Node t[], int root) {
71     // Keep output format valid even for empty trees.
72     if (root == -1) {
73         printf("\n");
74         return;
75     }
76
77     // Manual stack for preorder traversal.
78     int top = 0;
79     workStack[top++] = root;
80     // first controls spaces between numbers.
81     int first = 1;
82
83     while (top > 0) {
84         int cur = workStack[--top];
85         if (first) first = 0;
86         else printf(" ");
87         printf("%lld", t[cur].key);
88
89         // Push right first so left is processed first.
90         if (t[cur].right != -1) workStack[top++] = t[cur].right;
91         if (t[cur].left != -1) workStack[top++] = t[cur].left;
92     }
93
94     // End one traversal line.
95     printf("\n");
96 }
97
98 int main() {
99     int n1, n2;
100     int root1 = -1, root2 = -1;
101
102     // ===== Build T1 =====
103     // Read node count of first BST.
104     scanf("%d", &n1);
105     // Pass 1: read raw node data only.
106     for (int i = 0; i < n1; i++) {
107         scanf("%lld %d", &T1[i].key, &parent1[i]);
108         // Initialize child links before connection.
109         T1[i].left = -1;
110         T1[i].right = -1;
111     }
112     // Pass 2: connect parent-child relationships.

```

```

113     for (int i = 0; i < n1; i++) {
114         // Parent = -1 means this node is root.
115         if (parent1[i] == -1) {
116             root1 = i;
117         } else {
118             int p = parent1[i];
119             // Rule: equal keys must go to right subtree.
120             if (T1[i].key < T1[p].key) T1[p].left = i;
121             else T1[p].right = i;
122         }
123     }
124
125     // ===== Build T2 =====
126     scanf("%d", &n2);
127     // Same two-pass construction for second BST.
128     for (int i = 0; i < n2; i++) {
129         scanf("%lld %d", &T2[i].key, &parent2[i]);
130         T2[i].left = -1;
131         T2[i].right = -1;
132     }
133     for (int i = 0; i < n2; i++) {
134         if (parent2[i] == -1) {
135             root2 = i;
136         } else {
137             int p = parent2[i];
138             if (T2[i].key < T2[p].key) T2[p].left = i;
139             else T2[p].right = i;
140         }
141     }
142
143     // Read target sum N.
144     scanf("%lld", &N);
145
146     // ===== Prepare sorted keys =====
147     // Inorder traversal of BST is ascending.
148     int len1 = collectInorder(T1, root1, keys1);
149     int len2 = collectInorder(T2, root2, keys2);
150
151     // ===== Find all valid equations =====
152     for (int i = 0; i < len1; i++) {
153         // Skip duplicate A to avoid duplicate equations.
154         if (i > 0 && keys1[i] == keys1[i - 1]) continue;
155         long long a = keys1[i];
156         long long b = N - a;
157         // Binary search on sorted keys2.

```

```

158         if (findKey(keys2, len2, b)) {
159             resultA[resultCount] = a;
160             resultB[resultCount] = b;
161             resultCount++;
162         }
163     }
164
165     // ===== Print pair existence and equations =====
166     if (resultCount > 0) {
167         printf("true\n");
168         // Already sorted by A due to inorder sequence.
169         for (int i = 0; i < resultCount; i++) {
170             printf("%lld = %lld + %lld\n", N, resultA[i], resultB[i]);
171         }
172     } else {
173         printf("false\n");
174     }
175
176     // ===== Print required preorder traversals =====
177     printPreorder(T1, root1);
178     printPreorder(T2, root2);
179
180     // Normal program termination.
181     return 0;
182 }

```

Declaration

I hereby declare that all the work done in this project titled “Normal A+B with Binary Search Trees” is of my independent effort.